

COMPENG 2DX3 Project Final Report

April 7, 2026
Nihal Inel – inela - 400448431

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario. Submitted by **[Nihal Inel, inela, 400448431]**

Contents

(1)	Device Overview	2
1.1	Features	2
1.2	General Description	2
1.3	Block Diagram	3
(2)	Device Characteristics Table	3
(3)	Detailed Description	4
3.1	Distance Measurement.....	4
3.2	Visualization	5
(4)	Application Note, Instructions, and Expected Output	6
4.1	System Overview	6
4.2	Hardware Setup.....	6
4.3	Software Setup	6
4.4	Operating Procedures.....	7
4.5	Expected Output with Campus Location Scan	7
(5)	Limitations	9
(6)	Circuit Schematic.....	10
(7)	Programming Logic	11
	References.....	12

(1) Device Overview

1.1 Features

- 360 degrees planar distance scanning using the VL53L1X Time-of-Flight sensor mounted on a 28BYJ-48 stepper motor
- 32 distance measurements per rotation at 11.25° angular resolution
- I2C communication between microcontroller and ToF sensor at 100 kbps
- UART serial data transmission to PC at 115200 baud
- Distance and coordinate data stored in RAM as matrices during transmission, no onboard flash storage required
- MATLAB-based 3D point visualization with scatter plot and connected ring views
- Polling-based button input on PJ0 for scan start control between each movement
- Automatic 360° reverse unwind between scans to prevent wire tangling
- Three real-time status LEDs: PF0 for measurement status, PN0 for UART transmission status, PN1 for motor running status
- Configurable number of scans and displacement spacing via MATLAB script
- Operates at assigned bus speed of 14 MHz (PSYSDIV = 33) and 3.3V.
- Maximum sensing range of approximately 4 m in default long-range mode
- Tested at ITB hallway (Location B) with 20 scan positions at 20 cm spacing
- Cost of approximately \$200, including all hardware components

1.2 General Description

The ToF 3D Spatial Mapping System is a custom embedded data acquisition platform designed for indoor environment mapping. The system has a VL53L1X Time-of-Flight sensor mounted on a stepper motor to collect 360° distance measurements in a vertical plane at each scan position. The ToF sensor internally handles signal acquisition and analog-to-digital conversion of the reflected infrared photon signal, outputting a calibrated digital distance value in millimetres over I2C, which eliminates the need for external signal conditioning or ADC circuitry on the microcontroller [1]. The operator manually moves the sensor along a horizontal axis between scans, pressing a button to trigger each new rotation. Collected distance data is transmitted over UART to a PC where a MATLAB script maps the environment as a 3D point cloud.

The system is built around the MSP432E401Y microcontroller running at 14 MHz (PSYSDIV = 33 inside PLL.h), which is the assigned frequency. The microcontroller coordinates driving the stepper motor through Port H using a 4-phase half-step sequence, reading distance measurements from the ToF sensor over I2C on PB2/PB3, and transmitting formatted CSV data over UART0 at 115200 baud (assigned in class). Three status LEDs provide real-time feedback during operation. A polling-based button on PJ0 allows the operator to control scan initiation at each displacement position.

On the PC side, a MATLAB script opens the serial port, sends a start character to the microcontroller (set to ENTER), and reads incoming measurement data line by line. Each data point is converted from millimetres to metres and transformed from polar to Cartesian coordinates using trigonometric functions. The resulting 3D coordinates are displayed as both a scatter plot and a connected ring diagram.

The system is intended for use in indoor spaces with wall distances up to approximately four metres, making it suitable for hallway and room mapping applications.

1.3 Block Diagram

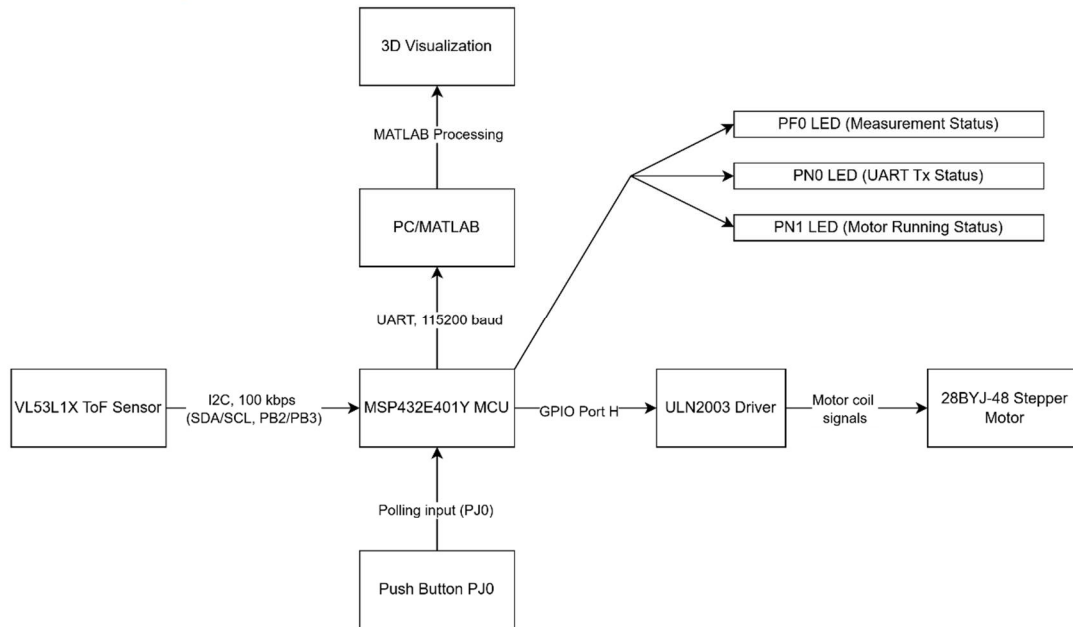


Figure 1: Data flow graph

(2) Device Characteristics Table

Parameter	Value
Microcontroller	MSP432E401Y
Assigned Bus Speed	14 MHz (PSYS DIV = 33)
ToF Sensor and Communication Protocol	VL53L1X and I2C
I2C Address	0x29
Clock Speed	100 kbps
Operating Voltage	3.3V
Distance Mode	Long Range (default)
Maximum Sensing Range	4 m
Angular Resolution	11.25° per measurement
Measurements per Rotation	32
Stepper Motor	28BYJ-48
Motor Driver	ULN2003
PC Communication Protocol	UART
UART Baud Rate	115200 bps
Measurement Status LED	PF0
UART Tx Status LED	PN0
Motor Running Status LED	PN1
Input Switch	PJ0
Implementation Mode	Polling
Data Visualization	MATLAB

Table 2: Device characteristics table

(3) Detailed Description

3.1 Distance Measurement

The distance measurement subsystem integrates the sensor, the motor, and the microcontroller to collect 32 distance samples per 360° rotation at 11.25° angular resolution.

At startup, the microcontroller initializes all peripherals including I2C, UART, and GPIO, then waits for the sensor to complete its boot sequence. Once booted, the sensor is initialized with default settings and ranging is started. The system then waits for two triggers before scanning begins: first a start character sent from MATLAB over UART, then a physical button press to begin the first scan.

For each scan, the motor rotates clockwise in 32 steps of 11.25° each. Each angular step is driven through a 4-phase coil sequence output through the GPIO port and amplified by the motor driver board. A settling delay follows each step before the sensor is polled for a data-ready signal. Once the sensor indicates a measurement is ready, the distance value in millimetres is retrieved over I2C. The measurement status LED flashes to confirm each reading was taken.

The distance value is then formatted as a comma-separated string containing the scan index, angle index, and distance in millimetres, and transmitted to the PC over UART. The UART transmission status LED flashes at the start of each scan's transmission block to provide visual confirmation that data is being sent. The motor running status LED remains on for the duration of the scan and turns off when the motor stops.

When the number of scans is more than one, the motor automatically reverses a full 360° to unwind the sensor wires, then waits for another button press before the next scan begins. There is also a timeout period set in MATLAB to allow for movement in between each scan. The operator manually advances the sensor to the next position before pressing the button. This process repeats until all scans are complete, at which point movement stops and the program is done.

The polling approach is used throughout. The button is checked by continuously reading the GPIO input register, and the sensor data-ready flag is checked by repeatedly querying the sensor over I2C until it confirms a measurement is available. This approach keeps the implementation straightforward while meeting all timing requirements for the application.

The stepper motor is the primary limitation on scan speed. Each angular step involves multiple delay calls to allow the motor coils time to physically move the rotor, accounting for the majority of time per scan. The sensor measurement and UART transmission complete quickly by comparison and do not have a significant contribution to overall scan duration.

3.2 Visualization

The visualization subsystem runs on the PC using MATLAB. Once the serial port is opened at 115200 baud, MATLAB sends a chosen start character to the microcontroller to trigger data transmission. Incoming data is read line by line, where each line contains three comma-separated values: scan index, angle index, and distance in millimetres. These are parsed and the distance is converted from millimetres to metres.

Each measurement is converted from polar to Cartesian coordinates. The angle for each measurement is the angle index multiplied by 11.25° , converted to radians. The horizontal and vertical spatial coordinates are calculated as:

$$Y = d \times \cos(\theta)$$

$$Z = d \times \sin(\theta)$$

where d is the distance in metres and θ is the angle in radians. For example, at angle 90° and distance 1.000 m: $Y = 1.000 \times \cos(90^\circ) = 0$ m, $Z = 1.000 \times \sin(90^\circ) = 1.000$ m. The displacement coordinate x is assigned based on the scan number, with a fixed spacing between consecutive scans set in the MATLAB script. Displacement refers to the physical distance the sensor assembly is moved along the x -axis between consecutive scans. In this implementation, displacement is set manually by the operator and represented in the visualization through the `x_spacing` variable, which assigns each scan ring a fixed x -coordinate offset. This places each scan ring at a distinct position along the x -axis to form the 3D spatial model.

All coordinates are stored in matrices of size (`num_scans` $\times$ `num_angles`). Two plots are generated side by side in a single figure window. The first is a 3D scatter plot where each point is colored by scan index using a graduated colormap, giving a visual overview of the complete point cloud. The second plot renders each scan as a closed polygon ring by connecting the 32 measurement points in angular order and appending the first point to close the loop. Consecutive scan rings are connected by lines at matching angle indices to show spatial continuity between positions.

The number of scans, angular measurements per scan, and displacement spacing between scans are all configurable directly in the MATLAB script through the variables `num_scans`, `num_angles`, and `x_spacing`. This allows the same system to be used for different environments and scan densities without modifying the microcontroller firmware. Only the MATLAB script needs to be updated to match the intended scan configuration.

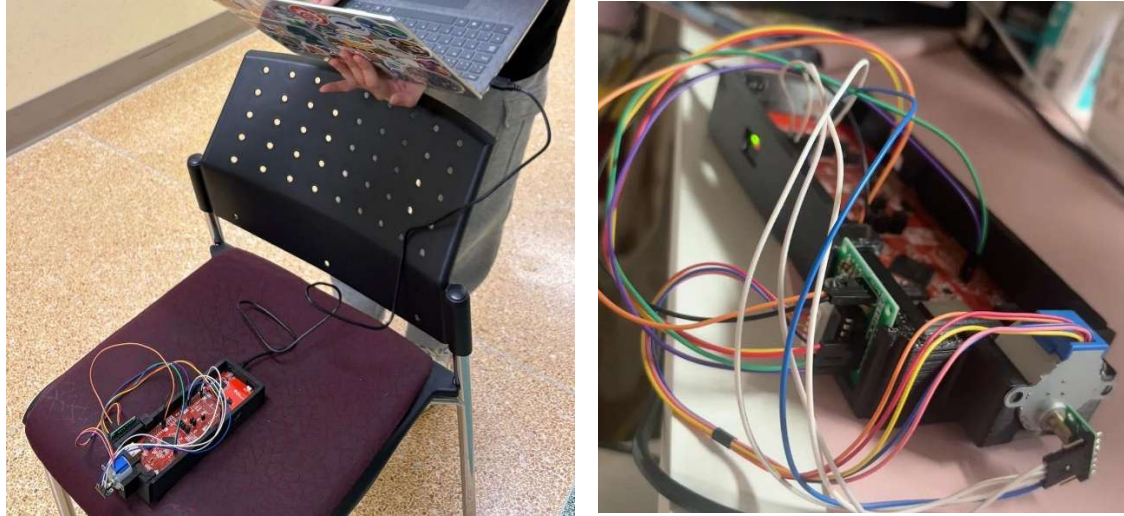
Axis limits are adjusted to match the scanned environment. For the ITB hallway scan, limits of ± 3 m on the Y and Z axes were used to capture the full extent of the space. Setting appropriate axis limits is important because limits inherited from template code designed for short-range tests will clip valid data points from larger environments.

The coordinate conversion relies on the trigonometric functions provided by MATLAB, which handles all floating-point arithmetic on the PC side. No trigonometric computation is performed on the microcontroller. The raw distance and angle index values are transmitted as integers and all conversion is done in MATLAB after reception.

(4) Application Note, Instructions, and Expected Output

4.1 System Overview

This section provides steps for the user on how to use/setup the ToF 3D Spatial Mapping System. Following these instructions will allow the user to collect distance measurements and generate a 3D visualization of an indoor environment without assistance. Figure below shows the assembled hardware setup used during scanning.



Figures 2 and 3: Assembled hardware setup

4.2 Hardware Setup

- Assemble the stepper motor and ToF sensor using the provided 3D printed bracket. The sensor is mounted on the side of the bracket which attaches to the motor shaft, allowing the sensor to rotate in a vertical plane during scanning.
- Connect the motor driver board to Port H on the microcontroller (PH0 through PH3) using jumper wires.
- Connect the ToF sensor to the microcontroller using two wires: SDA to PB3 and SCL to PB2. Also connect the sensor VCC to 3.3V and GND to GND on the board.
- Connect the microcontroller to the PC using a USB cable. The board will appear as a COM port in Device Manager.
- Verify the three status LEDs are functional: PF0 for measurement status, PN0 for UART transmission status, and PN1 for motor running status.

4.3 Software Setup

4.3.1 Keil Project Code:

- Open the Keil project in the provided source code folder.
- Verify that `num_scans` in `main.c` is set to 3 for the demonstration scan. If the desired number of scans are any other number, ensure it matches `num_scans` in MATLAB at all times.
- Build the project and flash it to the microcontroller.
- Once flashed, the onboard LEDs will flash briefly to confirm the ToF sensor has booted successfully.

4.3.2 MATLAB Code:

- Open MATLAB and open file: Studio9_E2_matlab_sol.m.
- Open Device Manager on Windows and expand the Ports section. Identify the COM port assigned to the microcontroller, which typically appears as "Application/User UART". Note the port number.
- Update the port variable at the top of the code file to match the identified COM port (for example, "COM5").
- Set num_scans = 3 (or any desired number, ensuring it matches the .c code in Keil at all times) and x_spacing = 0.10 (10 cm, or any other desired spacing number) in the configuration section of the script. Notice the more the number of scans and the less the spacing between each scan is better for the accuracy of the special scan.
- Set device.Timeout to desired number of seconds allowed in between each scan.

4.4 Operating Procedures

- Power on the microcontroller by connecting it to the PC via USB. Wait for the onboard LEDs to flash, indicating the sensor has booted.
- Run *clear device* command in the MATLAB Command Window to ensure a clean start.
- Run Studio9_E2_matlab_sol.m in MATLAB. The script will open the serial port and prompt the user to press Enter.
- Press Enter in the MATLAB Command Window. The script will send the start character to the microcontroller and wait for data.
- Press the onboard User Switch 1 (PJ0) on the microcontroller to begin the first scan. The PN1 LED will turn on to indicate the motor is running and PF0 will flash with each measurement taken.
- Once the first scan completes, the motor will automatically reverse to unwind the wires. Manually move the device 10 cm (or whatever the desired x_spacing value is) further in the intended direction of the scan. Wait for the motor to stop, then press PJ0 again to begin the second scan, before the timeout period is over
- Repeat previous step for the third scan. After the third scan completes the program halts automatically.
- MATLAB will display the 3D visualization once all data has been received.

4.5 Expected Output with Campus Location Scan

During scanning, the MATLAB Command Window will display each measurement as it is received in the following format:

```
Opening: COM5
Press Enter then press button on MCU...
Receiving data...
Scan 1, Angle 0.00 deg, Dist 0.0290 m
Scan 1, Angle 11.25 deg, Dist 0.0060 m
Scan 1, Angle 22.50 deg, Dist 0.0170 m
Scan 1, Angle 33.75 deg, Dist 0.0320 m
Scan 1, Angle 45.00 deg, Dist 0.0380 m
Scan 1, Angle 56.25 deg, Dist 0.0500 m
Scan 1, Angle 67.50 deg, Dist 0.0630 m
Scan 20, Angle 247.50 deg, Dist 0.7850 m
Scan 20, Angle 258.75 deg, Dist 0.2010 m
Scan 20, Angle 270.00 deg, Dist 0.1060 m
Scan 20, Angle 281.25 deg, Dist 0.1340 m
Scan 20, Angle 292.50 deg, Dist 0.6940 m
Scan 20, Angle 303.75 deg, Dist 0.7680 m
Scan 20, Angle 315.00 deg, Dist 0.3380 m
Scan 20, Angle 326.25 deg, Dist 0.2770 m
Scan 20, Angle 337.50 deg, Dist 0.2950 m
Scan 20, Angle 348.75 deg, Dist 0.2610 m
Closing: COM5>>
```

Figure 4: MATLAB Command Window output

Above figure is for $\text{num_scans} = 20$ and as can be seen, each angle is 11.25 degrees more than the previous angle.

Once all scans are complete, a figure window titled "G3 ToF 3D Scan" will open displaying two plots side by side. The left plot shows a 3D scatter plot of all measurement points colored by scan position. The right plot shows the same data rendered as closed polygon rings connected between scan positions. Figure 5 shows the hallway visualization alongside a photograph of the scanned location (Figure 6). Note that the outstanding points are the wires occasionally blocking the sensor for demonstration purposes. Ensure all wires are clamped together to avoid inaccurate distance values.

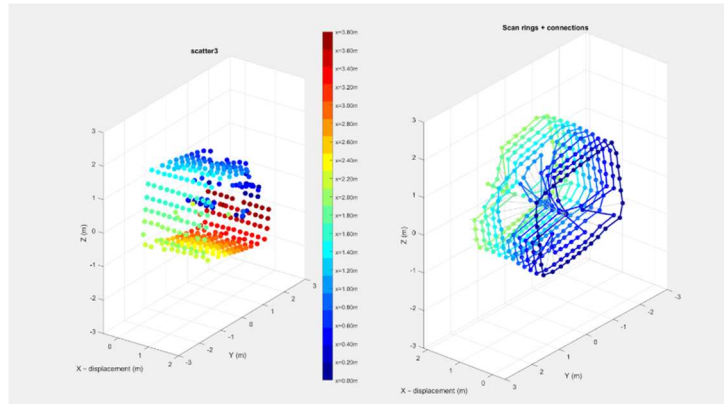


Figure 5: 3D Visualization



Figure 6: Photograph of the assigned hallway

The system was used to scan Location B, a hallway in the Information Technology Building (ITB) at McMaster University. Twenty scans were collected at 20 cm intervals along the hallway with the sensor placed on a chair at a consistent height for each scan position.

The 3D model captures the rectangular cross-section of the hallway, with consistent wall distances of approximately 1.7 m to 2.7 m across scans. Minor variations between scans are objects present in the hallway and slight sensor tilt between positions. Note that any outstanding outlier points in the scatter plot are caused by wires occasionally blocking the sensor. Ensure all wires are clamped together to minimize this effect.

(5) Limitations

- a. **Floating point and trigonometric functions:** The microcontroller has limited native floating point support. To avoid this limitation, all trigonometric computation and coordinate conversion is handled in MATLAB (PC side). The microcontroller transmits only raw integer values for scan index, angle index, and distance in millimetres. MATLAB then performs the $\cos()$ and $\sin()$ calculations using double precision floating point to convert polar measurements to Cartesian coordinates. The combination of limited floating point hardware and the need for trigonometric conversion means all coordinate computation must be offloaded to the PC, which introduces a dependency on the MATLAB environment for any spatial mapping output.
- b. **Maximum quantization error:** The ToF sensor operates with a range resolution of 1 mm in default mode. The maximum quantization error is therefore 0.5 mm, which at the maximum sensing range of 4 m corresponds to a relative error of 0.0125%.
- c. **Maximum serial communication rate:** The maximum UART baud rate implemented is 115200 bps (baudrate = 115200 in MATLAB), verified by confirming all data was received successfully without errors across scans.
- d. **Microcontroller to ToF sensor communication:** The ToF sensor (VL53L1X) communicates with the microcontroller (MSP432E401Y) over I2C at 100 kbps using SDA on PB3 and SCL on PB2.

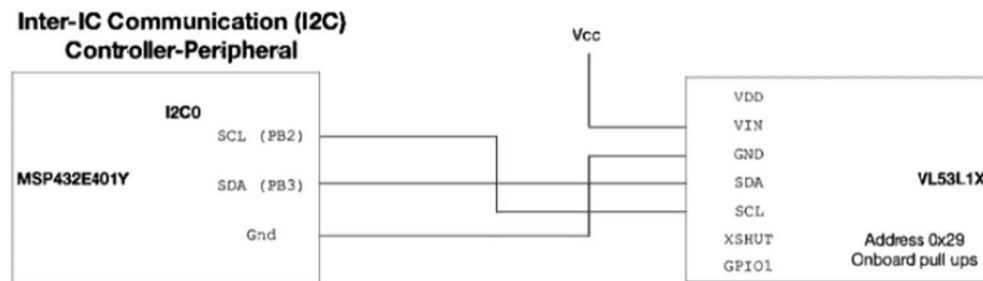


Figure 7: Wiring Layout [2]

- e. **Primary system speed limitation:** The stepper motor delays are the primary limitation on system speed. Each of the 32 angular steps requires 640 ms of motor delay, giving over 20 seconds of motor time per scan. The sensor measurement and UART transmission contribute negligibly by comparison.
- f. **Bus speed configuration:** Based on the student number, the assigned bus speed is 14 MHz. Using the formula $f_{\text{bus}} = 480 / (\text{PSYSDIV} + 1)$, PSYSDIV is set to 33, giving $480 / 34 = 14.12$ MHz. This is defined in PLL.h and applied in PLL.c during the PLL initialization sequence. The achieved bus speed was verified by toggling a GPIO pin and measuring the resulting frequency using the Analog Discovery 2. However, the limitation with the assigned frequency is the speed it results in. Therefore, for the purpose of faster scanning, PSYSDIV can be reduced to 3, increasing the bus speed to 120 MHz.
- g. **Status LED descriptions:** For a second least significant student number digit of 3: PF0 flashes after each distance measurement, PN0 flashes at the start of each scan's transmission block, and PN1 remains on while the motor is actively rotating and turns off when the motor stops.

(6) Circuit Schematic

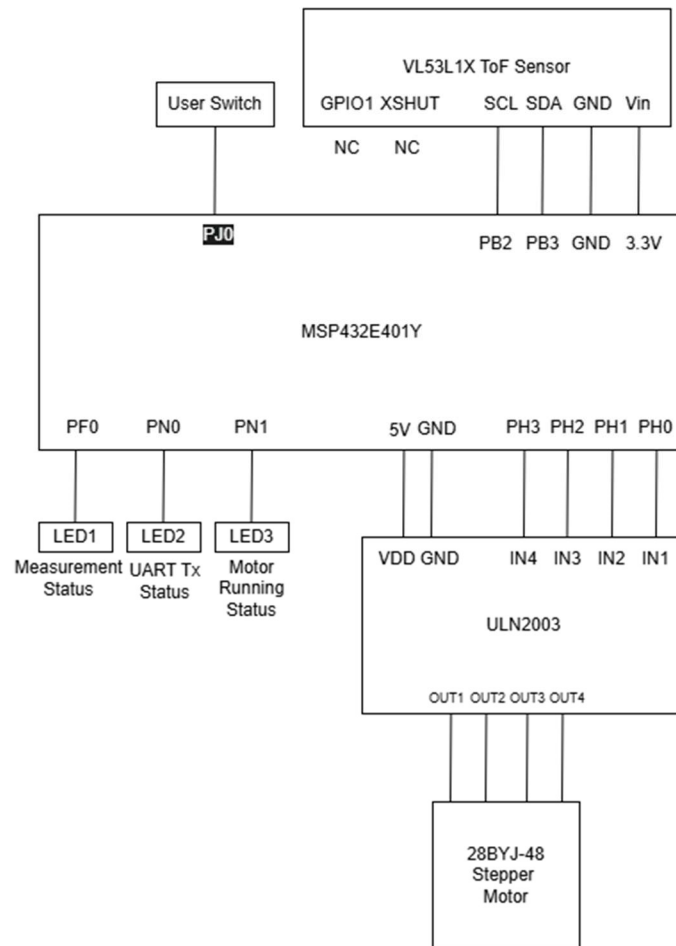


Figure 8: Circuit Schematic

(7) Programming Logic

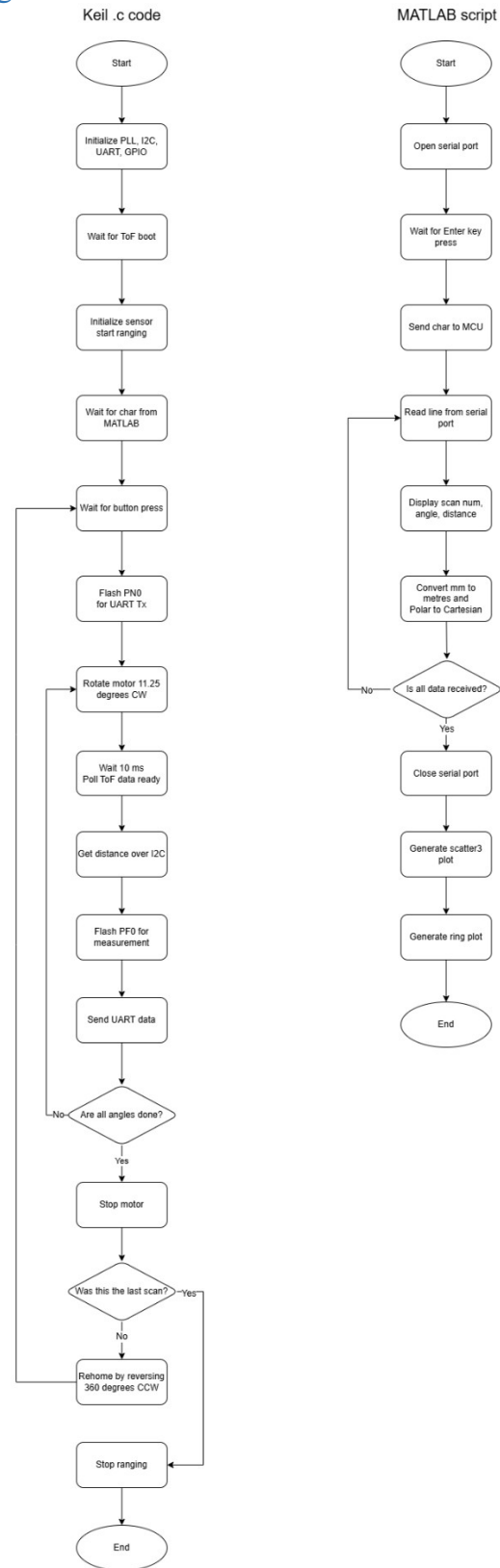


Figure 9: Firmware and Visualization Flowcharts

References

- [1] “This is information on a product in full production. VL53L1X A new generation, long distance ranging Time-of-Flight sensor based on ST’s FlightSense™ technology Datasheet -production data Features,” 2018. Available: <https://www.pololu.com/file/0J1506/vl53l1x.pdf>
- [2] S. Athar, Y. Haddara, and T. E. Doyle, 2025–2026 2DX3 Laboratory Manual, McMaster University, 2025.